

Member ID: _____

Time: _____

Rank: _____



JAVA PROGRAMMING

(340)

STATE 2023

PRODUCTION:

Crypto Exchange Portfolio Manager _____ (420 points)

Test Time: 90 minutes

GENERAL GUIDELINES:

Failure to adhere to any of the following rules will result in disqualification:

1. Member must hand in this test booklet and all printouts if any. Failure to do so will result in disqualification.
2. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests (handwritten, photocopied, or keyed) are allowed in the testing area.
3. Electronic devices will be monitored according to ACT standards.

You will have ninety (90) minutes to complete your work.

Your name and/or school name should *not* appear on work you submit for grading.

1. Create a folder on the flash drive provided using your contestant number as the name of the folder.
2. Copy your entire solution/project into this folder.
3. Submit your entire solution/project so that the graders may open your project to review the source code.
4. Ensure that the files required to run your program are present and will execute on the flash drive provided.

*Note that the flash drive letter may *not* be the same when the program is graded as it was when you created the program.

*It is recommended that you use relative paths rather than absolute paths to ensure that the program will run regardless of the flash drive letter. It is HIGHLY recommended that you place all of the files into one folder. Graders will not make any adjustments to your program.

The graders will *not* compile or alter your source code to correct for this.
Submissions that do *not* contain source code will *not* be graded.

Assumptions to make when taking this assessment:

- There will be 8 records (objects) created from the supplied txt files (9 total) that will be a combination of two different types of txt files being read: a list of Coins that stores their data, and then each coin has its own txt file to add more data.
- The program will automatically exit after the following: txt files have been read, the required objects created along with their nest objects, and the formatted output is produced in the console.
- All txt files for creating the objects in question have been provided. Given the complexity of the project, you will have access to a single file that will contain all of the classes needed to complete the task. It is not necessary to store object classes as their own file; however, if you choose to do so, there is no penalty.
- The output in console needs to be as close as possible to the example provided.

Development Standards:

- Your Code must use a consistent variable naming convention.
- All subroutines (if any), functions (if any), and methods (if any) must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any). If you create a class, then you must use Javadoc comments.

Crypto Exchange Portfolio Manager

In this challenge you will be required to create a software program that will allow a company to import text files (txt) that have been created with delimiters. These txt files contain a set of data for a client's portfolio of various positions within the cryptocurrency marketplace. There are two major classifications of investment products contained within the list: coins and tokens. For simplicity purposes, coins are typically the term to denote the native coin for the blockchain (a blockchain is a very complex database network that is distributed amongst nodes that connect to each other like a chain). These coins serve a multitude of functions on its native blockchain. Tokens are very similar, however it is tethered to the specific blockchain it was created for. People and companies create tokens so they can use the already developed infrastructure created by the blockchain. Every coin and token has its own specific information, such as the URL for its source code, website URL, market cap (for now just think of market cap as the total amount that can be sold), as well as tags (meta tags) that can be used by third parties when trying to put it on their market exchanges.

To complete this challenge, you will be required to import a txt file that has coins and tokens stored on it as text that is separated by a delimiter. When reading the file, the program will make no difference between what is a coin and what is a token: its goal is to read the file and store each record into an object that will be stored in a data structure to be used to print the information to the console. During the reading of the file, the program will need to get the symbol of the coin or token, and then use that information to read through files stored in the folder, attempting to find the file match for that particular coin/token, because that file contains specification information about the coin/token that is going to be stored along with each corresponding coin/token that is stored in the data structure. For example, if you are scanning the coins.txt file and it is reading in the information for Bitcoins whose symbol is BTC, the program will need to read through the folder to find the corresponding BTC.txt file to add this information to the record that is being created for Bitcoin in the data structure of objects. Every coin and token has a corresponding file that matches its symbol that contains extra information about it.

Once the files are read and stored successfully you will be required to print all of the coins and tokens to the console following a specific pattern and formatting the numerical values accordingly. Due to crypto currencies being made with computers, they can have an extensive amount of data stored in their numerical variables. For the market cap, it will be stored as a long value. The rational numbers can still be stored in a double, and you need not worry about any scientific notation for large decimal place values. All double values when printed will be required to have up to four decimal place values (the ten-thousandths, for example \$2.7700, or \$456.2356). You will also be calculating the percent change of the current price versus the purchase price: this will be formatted to only the hundredths place value (for example 15.65%).

Input

In this program there will be no manual user input; instead, the input will come from using the `BufferedReader` to through the text file and separate the records based upon the delimiter (in this program it is a semicolon). As the information is read, the data will be stored into an `ExchangeWrapper` object. The wrapper, which is a common tool in the world of crypto, will create the object holding the position data (NOTE: investments are typically called positions). The program will then transfer each position of the `ExchangeWrapper` class objects into a data structure. Remember, the corresponding block information from the symbol-based text file needs to be created as an object and added to its position before placing the `ExchangeWrapper` class objects into a data structure.

Output

The data structure that you use should print out all of the information for each position and its block information (this is the separate text file that is imported for each object based upon its symbol). The output will be sent to the console by accessing each `Positions` object's `getInfo ()` method. This method will provide the formatting for all of the double place values. Remember: all double values when printed will be required to have up to four decimal place values (for example \$2.7700, or \$456.2356). You will also be calculating the percent change of the current price versus the purchase price: this will be formatted to only the hundredths place value (for example 15.65% or -34.04%). It should be mentioned that the `BlockInfo` objects also have a `getInfo ()` method that will return the objects information. This will need to be utilized to get the proper output; and like the `Positions`, in this method is a good location to format the Market Cap to have commas and a \$ symbol (no decimals for the Market Cap).

The output is required to match the order of the imported text file. An example of the output format is provided below. It is required that each record is printed on three lines. In addition, each record needs to have a hanging indent for readability. In line fields need to be separated with a vertical-line (pipe) | symbol. There should be no pipe symbol at the end of each line, or at the beginning. The beginning of the print to console is also required to have the header "Positions" as seen below; it is not required to have any addition characters preceding or following the word as seen below (the '=' symbol is not required). There will be a total of 8 records printed to the console. The only order they need to be in, is the exact order in which they are found in the `Coins.txt` file. You are not required to sort them for ascending/descending order. They need to be printed in the same order as found in the file.

=====POSITIONS=====

Name: Bitcoin | Symbol: BTC | Quantity: 0.0005 | Current Price: \$48,912.0000

Average Price \$56,234.0000 | Purchase Price: \$59,343.0000 | Percent Change: -17.58%

Web URL: bitcoin.org | SourceCode URL: github.com/bitcoin | Market Cap: \$896,690,280,785 | Tag: PoW

Sample Run Example This is the exact order that should be printed (NOTE: these examples have formatting errors due to the margins of the document template. For example, for some of the positions the third line wraps around to fourth line, when printing to console this should not be an issue; however, if your console automatically wraps this action will not be ignored). Also take notice that the market cap is formatted for commas with NO decimal places: this is a requirement. Hard coding this output will NOT earn any credit.

=====POSITIONS=====

Name: Bitcoin | Symbol: BTC | Quantity: 0.0005 | Current Price: \$48,912.0000

Average Price \$56,234.0000 | Purchase Price: \$59,343.0000 | Percent Change: -17.58%

Web URL: bitcoin.org | SourceCode URL: github.com/bitcoin | Market Cap: \$896,690,280,785 | Tag: PoW

Name: Ethereum | Symbol: ETH | Quantity: 2.3450 | Current Price: \$4,323.0000

Average Price \$4,465.0000 | Purchase Price: \$3,738.0000 | Percent Change: 15.65%

Web URL: Ethereum.org | SourceCode URL: https://github.com/ethereum | Market Cap: \$453,972,230,585 | Tag: PoW

Name: Polygon | Symbol: MATIC | Quantity: 56.3400 | Current Price: \$2.3400

Average Price \$1.8400 | Purchase Price: \$0.3400 | Percent Change: 588.24%

Web URL: https://polygon.technology/ | SourceCode URL: https://github.com/maticnetwork/ | Market Cap: \$12,920,349,074 | Tag: BEP20

Name: Sandbox | Symbol: SAND | Quantity: 1,222.0000 | Current Price: \$5.8700

Average Price \$2.7700 | Purchase Price: \$2.7200 | Percent Change: 115.81%

Web URL: www.sandbox.game/en/ | SourceCode URL: NA | Market Cap: \$4,498,404,032 | Tag: ERC-20

Name: Cardano | Symbol: ADA | Quantity: 3,245.0000 | Current Price: \$1.3400

Average Price \$2.2100 | Purchase Price: \$2.0900 | Percent Change: -35.89%

Web URL: cardano.org/ | SourceCode URL: https://cardanoupdates.com/ | Market Cap: \$41,051,193,159 | Tag: DPoS

Name: Shiba-Inu | Symbol: SHIB | Quantity: 1,345,987.0000 | Current Price: \$0.0000

Average Price \$0.0001 | Purchase Price: \$0.0000 | Percent Change: 44.47%

Web URL: shibatoken.com/ | SourceCode URL: NA | Market Cap: \$18,256,090,722 | Tag: ERC-20

Name: Dogecoin | Symbol: DOGE | Quantity: 33,922.0000 | Current Price: \$0.1595

Average Price \$0.2343 | Purchase Price: \$0.2386 | Percent Change: -33.15%

Web URL: dogecoin.com/ | SourceCode URL: <https://github.com/dogecoin/dogecoin> | Market Cap: \$20,943,279,561 | Tag: BEP20

Name: Uniswap | Symbol: UNI | Quantity: 234.2000 | Current Price: \$14.7500

Average Price \$11.2300 | Purchase Price: \$5.8200 | Percent Change: 153.44%

Web URL: uniswap.org/blog/uni/ | SourceCode URL: NA | Market Cap: \$9,153,029,112 | Tag: ERC-20

Sample Run Hint #1: The information in the text files have spaces within the entries that are between the delimiters. When the program reads this information it will transfer the spaces back to the program. If the spaces are not removed, this could have detrimental effects on your ability to complete all of the required tasks, especially when working with quantitative values.

Sample Run Hint #2: Remember the market cap is larger than an integer (int) value.

Sample Run Hint #3: The **BlockInfo readBlockTextFiles(BufferedReader blockIO)** method is *almost* exactly like the **ExchangeWrapper readCoinTextFiles(BufferedReader coinsIO)** method

Requirements:

1. You must complete an application with the main class called **CryptoExchangePortfolioManager**. This class will create the required methods to read the **Coins.txt** file and amend the appropriate symbol-based text file to the **BlockInfo** object that will be stored in a corresponding **Positions** objects that will be centralized to an object called **ExchangeWrapper** that will used to place the objects into a data structure.
2. The **CryptoExchangePortfolioManager** will be the driver program that runs the data reading from the file, creating the objects and placing them into the data structure.
3. Formatting the numerical data for printing to the console will be accomplished within the **Positions** and **BlockInfo** classes.
4. Your contestant number must appear as a comment at the top of the main source code file.
5. The program will display the outputs as shown in the example on the previous page.

Solution and Project		
The project is present on the flash drive	_____	10 points
The projects main class is named CryptoExchangePortfolioManager	_____	10 points
The class helper method is named readCoinTextFiles(BufferedReader coinsIO)	_____	10 points
The class helper method is named readBlockTextFiles(BufferedReader blockIO)	_____	10 points
The class helper method is named BufferedReader getReader(String name)	_____	10 points
The Position object helper method is named getInfo() and returns a string value that returns the following: (1)Name, (2) Symbol, (3) Quantity, (4)Current Price (5) Average Price, (6) Purchase Price, (7) Percent Change AND the information from the BlockInfo object (8) Web URL, (9) SourceCode URL, (10) Market Cap, (11) Tag	_____	10 points
The Position object helper method is named percentChange() calculates the percent of change from purchase price to the current price. Hint here's one way to get it: $((\text{currentPrice} - \text{purchasePrice}) / \text{purchasePrice}) * 100$	_____	10 points
The BlockInfo object helper method is named getInfo() and returns a string value that returns the following: (8)Web URL, (9) SourceCode URL, (10) Market Cap, (11)Tag	_____	10 points
Program Execution		
The program runs from the USB flash drive	_____	10 points
<i>If the program does not execute, then the remaining items in this section receive a score of zero.</i>		
The program displays the printout based upon the required order of investment position types per the instructions	_____	30 points
The output has each field in the records separated by a pipe symbol with no line starting or ending with the (pipe or vertical line symbol)	_____	10 points
The output has a hanging indent for the second and third line.	_____	10 points
The Positions data for the output is the first and second line and is in the proper order: (1) Name, (2) Symbol, (3) Quantity, (4) Current Price (5) Average Price, (6) Purchase Price, (7) Percent Change	_____	10 points
The BlockInfo data for the output is the third line and is in the proper order: (8) Web URL, (9) SourceCode URL, (10) Market Cap, (11) Tag	_____	10 points
(3) Quantity is formatted for up to four decimal places	_____	10 points
(4) Current Price (5) Average Price, (6) Purchase Price is formatted for up to four decimal places, commas, and \$ symbol	_____	30 points
(10) Market Cap is formatted for with NO decimal places, and includes commas, and \$ symbol	_____	20 points

Source Code Review		
The source code is properly commented		
A comment containing the contestant number is present		10 points
Methods and code sections are commented		20 points
Method successfully uses try... catch for exception handling for BufferedReader getReader(String name) when creating the file based upon the passed parameter and returns the BufferedReader object to the coinsIO in the Main method. The error message should read: "The file doesn't exist."		30 points
Method successfully uses try... catch for exception handling for ExchangeWrapper readCoinTextFiles(BufferedReader coinsIO) to detect IOException errors; the error message should read "I/O Error Coin"		10 points
Method ExchangeWrapper readCoinTextFiles(BufferedReader coinsIO) extracts all data fields from text file (NOTE: double values must be parsed from String values)		10 points
Method ExchangeWrapper readCoinTextFiles(BufferedReader coinsIO) successfully calls BlockInfo readBlockTextFiles(BufferedReader blockIO) with proper text file based upon the symbol and stores the returned BlockInfo object. Be sure the readBlockTextFiles method has a try/catch exception with the error message read "I/O Error Block"		50 points
Method ExchangeWrapper readCoinTextFiles(BufferedReader coinsIO) successfully creates Positions object that is used as a parameter to construct an ExchangeWrapper object that is returned at the completion of the process.		40 points
In the Positions object class complete the String getInfo() method to print out the data fields from the object when called by returning a String in the proper order (remember items 8-11 are from the nested BlockInfo object): (1) Name, (2) Symbol, (3) Quantity, (4) Current Price (5) Average Price, (6) Purchase Price, (7) Percent Change (8) Web URL, (9) SourceCode URL, (10) Market Cap, (11) Tag		30 points
Total Points		/420 points

Files and Class Behaviors/Attributes: Help Information

Files Needed (10 total): CryptoExchangePortfolioManager.java (contains all of the classes); Coins.txt; ADA.txt; BTC.txt; DOGE.txt; ETH.txt; MATIC.txt; SAND.txt; SHIB.txt; UNI.txt

(NOTE: you will program methods in the following classes: CryptoExchangePortfolioManager, Positions, and BlockInfo)

public class CryptoExchangePortfolioManager Attributes (NOTE: all are provided in the Main method):

- `BufferedReader coinsIO`: retrieves and creates the file in the program to be read
- `List<ExchangeWrapper> cryptoPortfolio`: data structure suggestion to store the `ExchangeWrapper` objects
- `ExchangeWrapper wrappedPosition`: used to store returned objects that are passed into data structure

public class CryptoExchangePortfolioManager Behaviors:

- **PROVIDED**: `public static void main (String args [])`: the base of the project is provided for you in this method. It will setup the order of the recommended processes such as calling a working sequence of methods to successfully extract all of the information from the text files, store it in a data structure and then print it to the console. You are not required to keep it intact, however the parts of the project that are required per the rubric will need to be included and easily identifiable.
- **ACTION**: `private static BufferedReader getReader(String name)`: you will create this method that will create the file based upon the String that is passed to this function. You will need to make sure there is a try/catch to handle a situation where the file does not exist (this will be tested by the graders).
- **ACTION**: `private static ExchangeWrapper readCoinTextFiles(BufferedReader coinsIO)`: you will create this method that will read the file based upon the `BufferedReader` object created by the `getReader(String name)` method. You will need to make sure there is a try/catch to handle a situation where there is an error in reading the lines of text.
- **ACTION**: `readBlockTextFiles(BufferedReader blockIO)`: you will create this method that will read the text file passed to it via the `BufferedReader` object. Note, you will need to use the `getReader` method to help in this process. In addition, this method is similar to the `readCoinTextFiles()` method but it will be handling different fields of information for `BlockInfo` object.

class `ExchangeWrapper`: this entire class is provided for you. It is important to familiarize yourself with its behaviors and attributes.

class `Positions`: this class is partially given to you. Here are the attributes that are **PROVIDED**:

- `private String type`: This is not used
- `private String name`: Name of the position
- `private String symbol`: All caps symbol for the position.

- private double quantity: How much is own (this is a rational number)
- private double currentPrice: The current trading price of the position
- private double avgCost: What has been its historical cost
- private double buyPrice: How much did the user pay for it
- public BlockInfo blkInfo: Information from another object that is tied to this position
- private double percentChange: Used in calculating the change in price by %

class Positions Behaviors:

- PROVIDED: Constructor: public Positions(type, name, symbol, quantity, currentPrice, avgCost, buyPrice, blkInfo) It should be noted that the actual constructor will use local variables that reflect the names of the instance variables in an abbreviated format.
- ACTION: public String getInfo(): You will create this method based upon the output requirements.
- ACTION: private String percentChange(): You will create this method based upon the requirements for calculating the percent of change. This method has been premade to reflect a format that will pass the change back as a String; however, you can alter this method to return a numerical value.

class BlockInfo: this class is partially given to you. Here are the attributes that are PROVIDED:

- private String website: website URL
- private String sourceCode: website URL if applicable
- private long marketCap: market cap total value
- private String tag: a metatag

class BlockInfo Behaviors:

- PROVIDED: Constructor: public BlockInfo(website, sourceCode, marketCap, tag) It should be noted that the actual constructor will use local variables that reflect the names of the instance variables in an abbreviated format.
- ACTION: public String getInfo(): You will create this method based upon the output requirements.